

Projeto re/Start

1. Identificação do Problema

Ideia do Projeto: Biblioteca Virtual Comunitária

Descrição do Problema:

A desigualdade no acesso à leitura é agravada pela escassez de bibliotecas físicas e pelo alto custo dos livros, excluindo grande parte da população para ter acesso ao conhecimento literário e técnico, consequentemente trazendo outras problemáticas em determinados âmbitos da vida do indivíduo.

Stakeholders:

- Leitores
- Editoras
- Escritores independentes
- Curadoria pedagógica
- AWS
- Escolas e universidades públicas

Justificativa:

Diante desse cenário, surge o desafio de criar uma biblioteca virtual que supere essas barreiras geográficas e financeiras através de uma plataforma acessível, transformando a tecnologia em uma ferramenta eficaz de democratização educacional e inclusão social.

A implementação de um sistema que permita aos estudantes a possibilidade de acessar livros de forma gratuita. Garantindo o acesso à cultura de forma democrática.

A plataforma proposta permitiria:

- **Acesso Igualitário:** Eliminar barreiras geográficas e econômicas ao acesso a livros
- **Desenvolvimento Intelectual:** Ampliar oportunidades de aprendizado contínuo
- **Inclusão Social:** Promover igualdade educacional em comunidades periféricas

- **Sustentabilidade Cultural:** Apoiar autores independentes e garantir acesso a obras clássicas e contemporâneas
- **Alinhamento com ODS:** Contribuir com os Objetivos de Desenvolvimento Sustentável da ONU, especialmente ODS 4 (Educação de Qualidade) e ODS 10 (Redução das Desigualdades)

2. Levantamento de Requisitos

Requisitos Funcionais:

Busca e Descoberta de Livros

- O sistema deve permitir buscar livros por título, autor, gênero, ano de publicação e palavras-chave
- O sistema deve classificar livros por categorias e subcategorias
- O sistema deve permitir filtrar por idioma, formato PDF e nível de dificuldade (MVP)
- (Fase 2+) Suportar múltiplos formatos (EPUB, etc.) e filtrar por formato

Leitura e Gerenciamento de Conteúdo

- O sistema deve permitir ler livros online com visualizador integrado
- O sistema deve permitir baixar livros em formatos PDF
- (Fase 2+) Suportar múltiplos formatos (EPUB, etc.)

Gestão de Conteúdo para Administradores

- O sistema deve permitir upload de novos livros com metadados (título, autor, sinopse, capa)
- O sistema deve permitir gerenciar direitos autorais e licenças de livros
- O sistema deve gerar relatórios de uso e estatísticas de acesso

Requisitos Não Funcionais:

Performance

- O sistema deve carregar páginas em menos de 3 segundos
- O sistema deve suportar leitura fluida mesmo em conexões 3G/4G
- O sistema deve permitir leitura offline após download do livro

Segurança

- O MVP utiliza apenas domínio público, portanto sem necessidade de DRM
- O sistema deve proteger direitos autorais através de Digital Rights Management (para fase 2+, quando adicionarmos obras com direitos autorais com autorização)
- O sistema deve cumprir com LGPD desde o MVP:
 - Política de privacidade clara sobre coleta de dados de download
 - Sem compartilhamento de dados com terceiros sem consentimento

Acessibilidade

- O sistema deve ser compatível com leitores de tela (WCAG 2.1 AA)
- O sistema deve permitir ajuste de tamanho de fonte e tema escuro/claro
- O sistema deve ser acessível via dispositivos móveis (iOS e Android) e desktop
- O sistema deve suportar navegação por teclado

Escalabilidade (fase 3+, após validação do MVP)

- O sistema deve suportar até 100.000 usuários simultâneos
- O sistema deve armazenar biblioteca com pelo menos 10.000 títulos inicialmente
- O sistema deve escalar automaticamente conforme crescimento da base de usuários

Disponibilidade

- O sistema deve garantir 99.9% de uptime
- O sistema deve ter backup automático de dados diariamente
- O sistema deve ter plano de recuperação de desastres

MVP (Produto Mínimo Viável):

Fase 1 (Lançamento Inicial)

- O sistema deve suportar até 500 usuários simultâneos.
- Catálogo de 500 livros de domínio público (clássicos da literatura brasileira, universais e Livros do BNCC (livros de acesso público)).
- Visualizador de livros online (PDF)
- Busca por título, autor, gênero, ano de publicação e palavras-chave.
- Download de livros em PDF
- Interface responsiva (mobile e desktop)
- LGPD compliance obrigatório (política de privacidade e consentimento)

3. Planejamento Ágil

Backlog de Tarefas:

1. Infraestrutura AWS serverless (baixo custo, Lambda em Python)

- Criar conta AWS, configurar orçamento e alarmes de custo (AWS Budgets / Cost Explorer) para não ultrapassar um limite mensal.
- Criar usuário e roles IAM com permissões mínimas para:
 - Execução das funções Lambda (apenas CloudWatch Logs, acesso ao RDS e ao S3).
 - Deploy da infraestrutura (API Gateway, Lambda, RDS, S3).
- Criar buckets Amazon S3 para:
 - Frontend estático (HTML/CSS/JS).
 - Armazenamento dos PDFs dos livros (ex.: bucket biblioteca-pdfs).

- Criar uma **API HTTP** no Amazon API Gateway para expor as rotas REST:
 - GET /books – listar livros.
 - GET /books/search – buscar por título, autor, gênero, ano, palavra-chave.
 - POST /downloads – registrar download.
 - POST /admin/books – criar/editar livro (rota protegida).
- Criar funções **AWS Lambda** com runtime **Python 3.x** para toda lógica de negócio:
 - Função list_books (Python) consulta o banco RDS e devolve JSON.
 - Função search_books (Python) monta queries SQL com filtros simples.
 - Função register_download (Python) insere registros na tabela downloads.
 - Função admin_upsert_book (Python) insere/atualiza registros na tabela books.
- Provisionar um banco de dados relacional pequeno no **Amazon RDS** (PostgreSQL ou MySQL) em uma instância db.t4g.micro dentro do free tier.
- Configurar a conexão das Lambdas Python com o RDS usando driver padrão (psycopg2 para Postgres ou PyMySQL para MySQL).

2. Backend serverless (API Gateway + Lambda Python + RDS)

- Implementar as funções Lambda Python:
 - list_books: retorna lista de livros da tabela books.
 - search_books: aplica filtros por título, autor, gênero, ano, palavras-chave (SQL com índices básicos).
 - register_download: insere registros na tabela downloads para gerar estatísticas de uso.
 - admin_upsert_book: cria/atualiza livros (título, autor, gênero, ano, nível BNCC, URL do PDF no S3).

- Configurar as integrações do API Gateway com as Lambdas:
 - GET /books → Lambda list_books.
 - GET /books/search → Lambda search_books.
 - POST /downloads → Lambda register_download.
 - POST /admin/books → Lambda admin_upsert_book (rota protegida por header ou API key).
- Criar o modelo de dados mínimo no RDS:
 - Tabela books (id, título, autor, gênero, ano, nível BNCC, url_pdf_s3).
 - Tabela downloads (id, book_id, data/hora, origem).
- Armazenar credenciais do banco (host, usuário, senha) como variáveis de ambiente nas Lambdas, evitando hard-code.

3. Frontend estático em S3

- Criar interface web simples (React ou HTML/CSS/JS):
 - Página inicial com lista de livros via GET /books.
 - Página de busca com filtros, chamando GET /books/search.
 - Tela de leitura, abrindo o PDF hospedado no S3 (URL vinda do backend).
- Configurar o bucket S3 como **Static Website Hosting** para servir o frontend.
- Integrar o frontend à API Gateway, usando a URL pública da API.
- (Opcional) Colocar **CloudFront** na frente do S3 para cache global e melhor desempenho, ainda aproveitando o free tier.
- Garantir responsividade (desktop e mobile).

4. Catálogo de livros e operação

- Fazer upload de ~500 livros em PDF para o bucket S3 dedicado (ex.: biblioteca- pdfs).
- Cadastrar metadados dos livros no banco RDS:
 - Script Python local chamando a API admin (POST /admin/books).
 - Ou inserção inicial direta na tabela books.
- Criar tela/endpoint mínimo de administrador para:
 - Cadastrar/editar livros.
 - Alterar metadados (autor, gênero, ano, nível BNCC, URL PDF).
- Registrar downloads via POST /downloads para relatórios simples de uso.

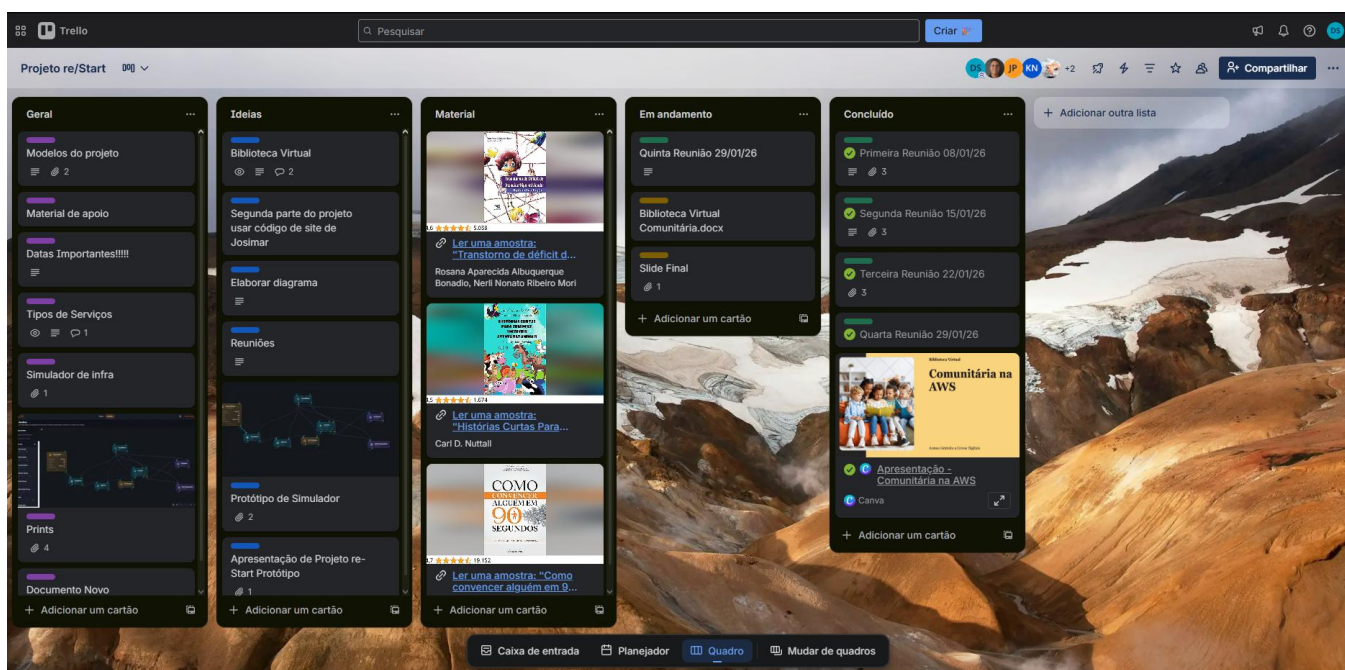
5. Testes, monitoramento e otimização de custos

- Testar performance (tempo de resposta da API, tempo de carregamento de PDFs).
- Configurar **Amazon CloudWatch** para monitorar erros e consumo:
 - Logs automáticos das Lambdas Python.
 - Métricas de latência/erros do API Gateway.
- Revisar **Cost Explorer** e AWS Budgets para garantir que:
 - Uso está dentro do **free tier** sempre que possível.
 - RDS está no menor tamanho viável.
- Ajustar configurações (memória/timeout das Lambdas, cache HTTP no frontend) para reduzir custo e manter boa experiência.

Sprints:

- **Sprint 1:** Infraestrutura AWS serverless com Python
 - Montar base S3 + API Gateway + Lambda (Python) + RDS.
- **Sprint 2:** Backend serverless (API Gateway + Lambda Python + RDS)
 - Implementar APIs de catálogo, busca e registro de download.
- **Sprint 3:** Frontend estático em S3
 - Criar interface web e integrá-la à API.
- **Sprint 4:** Catálogo real e painel simples de administração
 - Subir ~500 PDFs no S3 e disponibilizar painel para cadastrar/editar livros.
- **Sprint 5:** Testes, monitoramento e otimização de custos
 - Validar qualidade, medir desempenho e ajustar uso para manter custo baixo.

Quadro (Board):



- **Resultado esperado:**

- Endpoint público do API Gateway funcionando com Lambda Python.
- RDS ativo e acessível pela VPC onde as Lambdas serão configuradas (se estiverem em VPC).
- Estrutura serverless básica (Python) validada e com custo previsto dentro do free tier.